

Eduardo Lamounier, Heitor Freitas e Adrian da Silva

Campo Minado Infinito

Programação Orientada a Objetos

Belo Horizonte

2026

1 Introdução

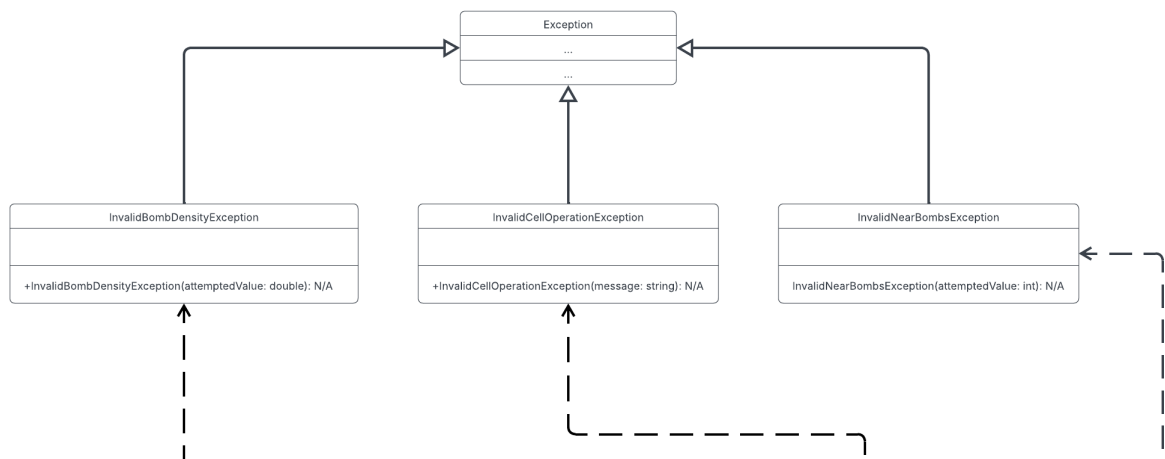


Figura 2 – Exceções da UML

As exceções personalizadas são herdadas da classe **Exception** e cada uma delas representam um estado de erro do programa. Cada uma delas tem um relacionamento de dependência, pois as classes dependentes utilizam elas para lançarem erros e mudar as exceções acarretaria mudanças nas operações das classes dependentes. Assim, a classe **MineField** depende de **InvalidBombDensityException**, **InvalidCellOperationException** tem uma relação com a classe abstrata **Cell** e **InvalidNearBombsException** com **SafeCell**.

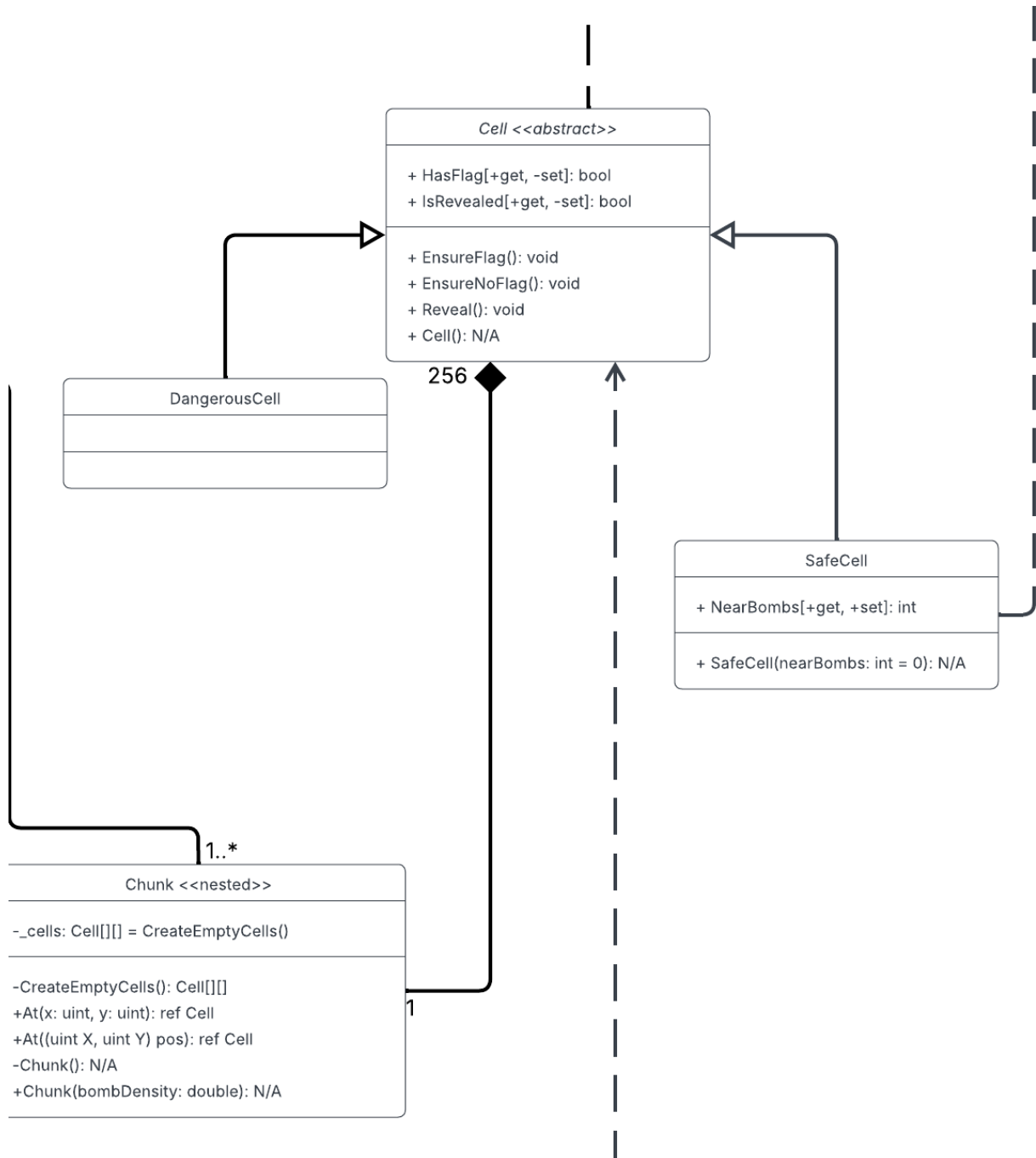


Figura 3 – Celula-UML

A classe **Cell** é uma classe abstrata, apesar de seu nome estar em itálico, para uma melhor visualização, o *abstract* foi colocado entre dois pares de sinal de menos '<' e maior '>'. Suas classes filhas são **SafeCell** e **DangerousCell**.

A **Chunk** é uma área 16x16 (256u.a.) de células do mapa, por isso a classe **Cell** tem um relacionamento de composição com **Chunk**, sendo essa última a classe contenedora (o todo).

Além do mais, a `Cell` faz uma relação de dependência com a classe `MineFieldRenderer`, pois essa utiliza a célula para fazer as suas operações, mas não guarda como um campo, por isso que uma mudança na classe resulta em mudanças nos métodos do dependente.

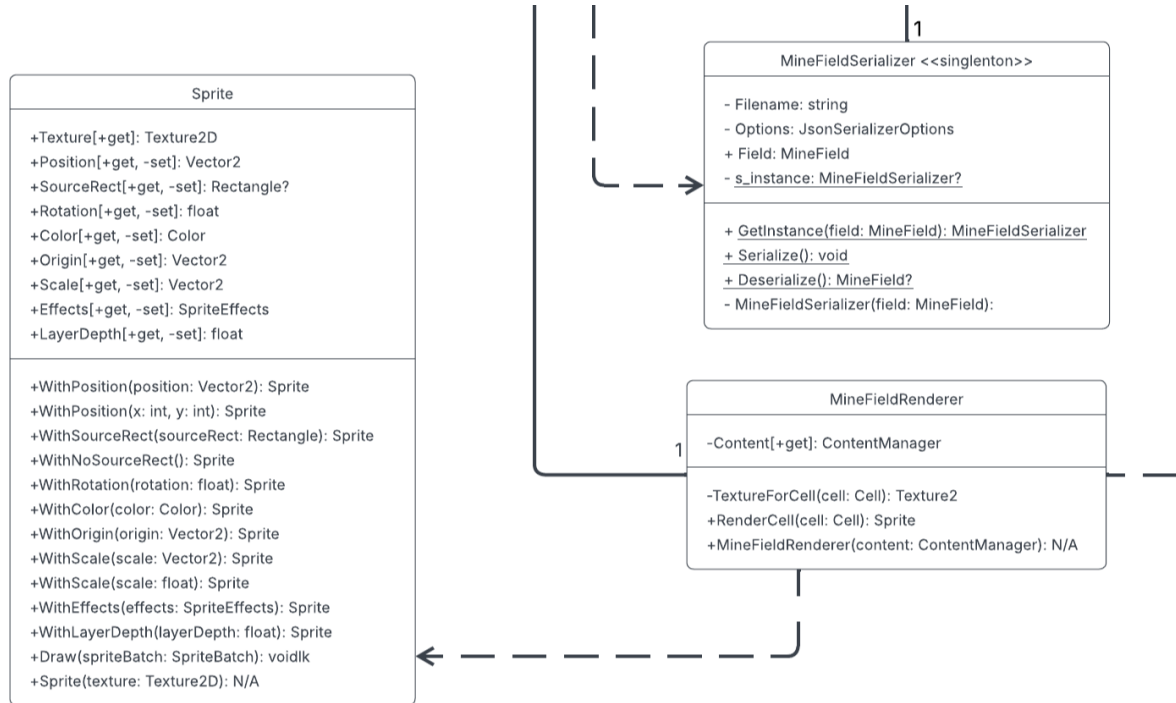


Figura 4 – renderização no UML

`MineFieldRenderer` é responsável por receber células do campo e retornar o sprite com textura correspondente ao estado e tipo da célula. Isso configura a relação entre `MineFieldRenderer` e `Sprite` como uma relação de dependência, já que a primeira depende a última para fazer sentido e, assim como no caso de `Cell`, nenhum objeto da classe é armazenado em `MineFieldRenderer`, o que significa que não se configura uma relação de composição.

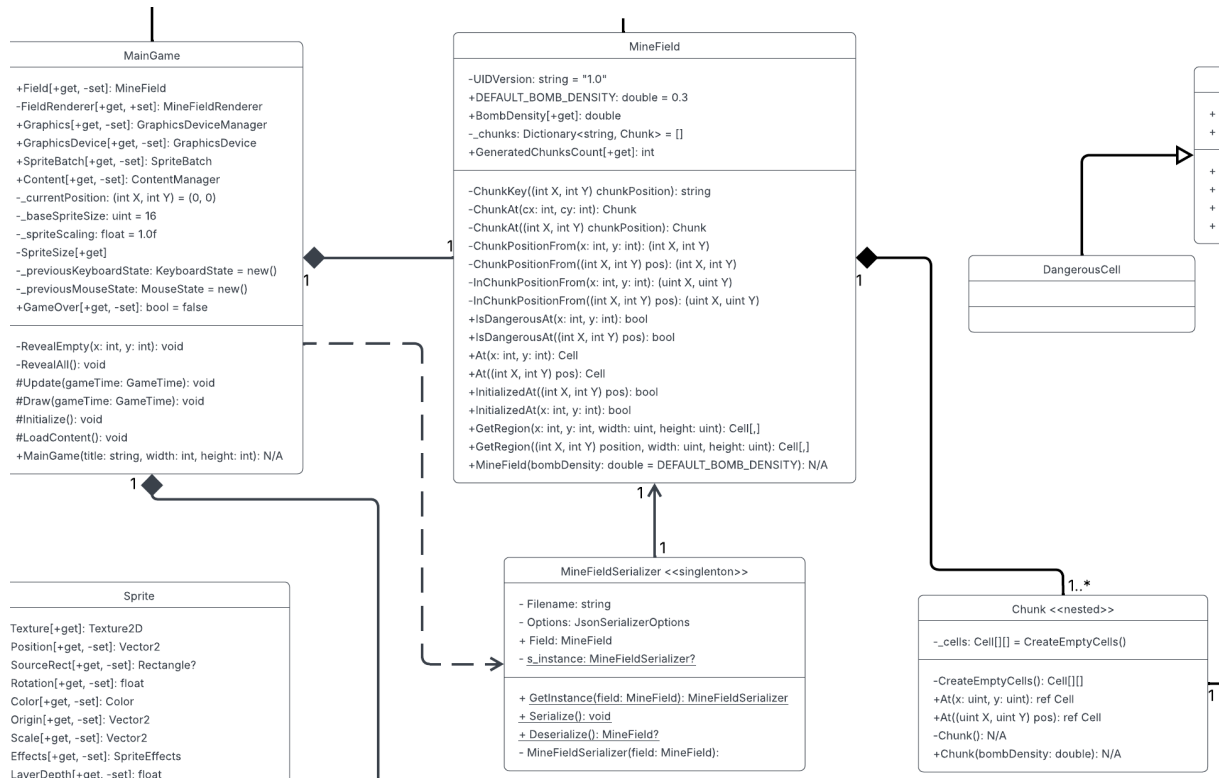


Figura 5 – Relacionamentos principais - UML

MainGame se trata da classe principal do programa, que é filha da classe **Game** que é a classe principal do **MonoGame**. Logo, para armazenar o estado do campo na sessão atual do jogo, ela contém uma propriedade **Field** que armazena uma instância de **MineField**. Isso configura essa relação como uma relação de composição - já que **MainGame** é composta por **MineField**.

MineField, de forma similar, armazena dentro da sua definição um dicionário com as diferentes *chunks* carregadas durante o jogo, se configurando também como uma relação de composição. Porque **Chunk** serve apenas para implementação da lógica geral do campo, ela é definida como classe privada dentro de **MineField** para encapsulamento da lógica interna do campo, e isso foi simbolizado no UML com "«nested»".

A classe **MineFieldSerializer** é responsável pela serialização do mapa e é um padrão de projeto denominado *Singleton*, como visto entre dois pares de símbolos de menor '<' e maior '>'. Por possuir um atributo, que é um objeto da classe **MineField**, mas o mesmo não ter uma dependência dele, pode se configurar que a relação entre eles é de associação direta, sendo possível associar do serializador para o campo minado mas o contrário não podendo ser viável, assim, para cada objeto do serializador possuir 1 objeto do **MineField**. Ademais, o **MainGame**, utiliza os métodos estáticos do serializador, mas não guarda um objeto dele, com isso, configurasse em uma relação de dependência, em que o dependente é a classe principal.

A classe `MineFieldRenderer`, por sua vez, tem uma relação de composição com `MainGame` visto que essa última armazena dentro de si um renderizador próprio - `FieldRenderer` - para gerar os sprites das células.

3 Conceitos aplicados

Aqui vamos mostrar os conceitos da disciplina que foram trabalhados nesse projeto e **como** eles foram aplicados.

3.1 Herança

Nós usamos herança, mais notavelmente, em dois contextos:

O primeiro foi na classe `MainGame` - classe principal do programa - onde definimos ela para herdar da classe `Game`, definida pelo Monogame. Isso é feito para poder implementar os métodos de renderização, atualização de estados etc que são necessárias para rodar o programa nesse framework.

```
1 // Em src/MainGame.cs:
2 namespace CampoMinado;
3
4 /// <summary>
5 /// Classe principal do programa. Contém a definição do método de
6 /// entrada e a lógica do game loop.
7 /// </summary>
8 public class MainGame : Game { ... }
```

O código acima mostra a definição de `MainGame` e a relação de herança entre essa classe e `Game`, do Monogame.

O segundo contexto onde herança foi utilizada foi nas classes que representam as células do campo minado. Para representar as células "perigosas" e "seguras", nós definimos as classes `SafeCell` e `DangerousCell` para herdar de uma classe abstrata `Cell`, que também contém funcionalidade comum entre todo tipo de célula.

```
1 // Em src/Cell.cs:
2 namespace CampoMinado.Core;
3
4 /// <summary>
5 /// Representa uma célula genérica ou não inicializada do campo
6 /// minado.
7 /// </summary>
8 [JsonDerivedType(typeof(SafeCell), "safe")]
9 [JsonDerivedType(typeof(DangerousCell), "dangerous")]
10
```

```
9 public abstract record class Cell { ... }
```

```
1 // Em src/SafeCell.cs:
2 namespace CampoMinado.Core;
3
4 /// <summary>
5 /// Representa uma célula sem bomba do campo minado. Armazena a
6   quantidade células com
7   bomba adjacentes.
8 /// </summary>
9 public record class SafeCell : Cell { ... }
```

```
1 // Em src/DangerousCell.cs:
2 namespace CampoMinado.Core;
3
4 /// <summary>
5 /// Representa uma célula com bomba do campo minado.
6 /// </summary>
7 public record class DangerousCell : Cell { ... }
```

Os códigos acima mostram as definições de `Cell`, `SafeCell`, `DangerousCell` e a relação de herança entre essas classes.

3.2 Polimorfismo

O conceito de polimorfismo foi trabalhado na classe `MainGame`, porque diferentemente da árvore de herança contendo as classes `Cell`, `SafeCell` e `DangerousCell`, `MainGame` não herda de `Game` apenas para ganhar funcionalidade, mas também para **definir** funcionalidade. A classe `MainGame` implementa métodos como `Update(...)`, `Draw(...)` e `Initialize(...)` através de sobrescrita de métodos - o que em C# necessita o uso da keyword `override`.

```
1 // Em src/MainGame.cs:
2 namespace CampoMinado;
3
4 /// <summary>
5 /// Classe principal do programa. Contém a definição do método de
6   entrada e a lógica do game loop.
7 /// </summary>
8 public class MainGame : Game
9 {
10     /// <summary>
```

```
10  /// Método chamado a todo frame. Contém lógica do programa que
    tem que ser
11  /// atualizada a todo momento.
12  /// </summary>
13  /// <remarks>
14  /// NÃO é responsável pela lógica de renderização, mas chama
    automaticamente o
15  /// método 'Draw', que tem.
16  /// </remarks>
17  protected override void Update(GameTime gameTime) { ... }
18
19  /// <summary>
20  /// Método chamado a todo frame. Contém lógica de renderização
    do programa.
21  /// </summary>
22  /// <remarks>
23  /// NÃO é responsável pela lógica periódica genérica do
    programa, que é responsabilidade
24  /// de 'Update'.
25  /// </remarks>
26  protected override void Draw(GameTime gameTime) { ... }
27
28  /// <summary>
29  /// Contém a lógica de inicialização do programa. É chamada
    antes do game loop com
30  /// 'Update' e 'Draw'.
31  /// </summary>
32  /// <remarks>
33  /// NÃO é responsável pela lógica de inicialização da classe
    nem pela lógica de
34  /// carregamento dos sprites, texturas etc (assets em geral).
35  ///
36  /// Chama 'LoadContent' - que é responsável pelo carregamento
    de assets -
37  /// automaticamente.
38  /// </remarks>
39  protected override void Initialize() { ... }
40
41  /// <summary>
42  /// Contém a lógica de carregamento dos assets do programa. É
    chamada antes do game
43  /// loop, depois da lógica de inicialização genérica - '
```

```

        Initialize'.
44     /// </summary>
45     protected override void LoadContent() { }
46 }

```

O código acima mostra a definição dos métodos de `MainGame` via sobrescrita de método. A implementação desses métodos, a definição das propriedades da classe e outros métodos que não tem a keyword `override` foram ocultados(as) para simplificar a leitura do código.

3.3 Classe Abstrata

A classe `Cell` é uma classe abstrata definida no nosso jogo. Essa decisão de design foi feita para garantir funcionalidades em comum entre os dois tipos existentes de célula, mas de forma a mostrar que uma instância da classe `Cell` não faz sentido. Essa classe serve apenas para ser a base da árvore de herança entre `SafeCell` e `DangerousCell`.

Como visto também na seção 3.1, a definição de `Cell` tem a keyword `abstract`, simbolizando o fato dela ser um "blueprint" para outras classes.

3.4 Collections

A estrutura de dados mais notável que foi utilizada no código foi a classe `Dictionary` do C#, que é um equivalente ao `HashMap` do Java. Essa estrutura foi usada em `MineField` para armazenar as chunks que foram carregadas de acordo com suas posições (x, y) no grid e poder acessá-las em tempo constante $O(1)$.

```

1  // Em src/MineField.cs:
2  namespace CampoMinado.Core;
3
4  /// <summary>
5  /// Representa o mapa do campo minado. Armazena o estado de todas
6  /// as células inicializadas
7  /// do mapa, encapsulando todos os detalhes de implementação.
8  /// </summary>
9  public class MineField
10 {
11     /// <summary>
12     /// Armazena as chunks inicializadas - ou seja, aquelas na quão
13     /// tiveram alguma célula
14     /// interagindo com o jogador.
15     /// </summary>
16     /// <remarks>

```

```

15  /// Usa uma string como chave para permitir serialização.
16  /// </remarks>
17  [JsonInclude]
18  private Dictionary<string, Chunk> _chunks = [];
19
20  /// <summary>
21  /// Converte uma coordenada do mapa para uma string que sirva
22  /// como
23  /// chave para <see cref="_chunks"/>
24  /// </summary>
25  private static string ChunkKey((int X, int Y) chunkPosition) =>
    $"{chunkPosition.X},{chunkPosition.Y}";

```

O código acima mostra a definição da propriedade privada `_chunks`, que é um `Dictionary` que armazena as chunks do mapa. Da mesma forma que em outros snippets de código, muitos detalhes de `MineField` foram omitidos aqui.

3.5 Arquivos

O estado do campo minado é salvo em um arquivo `JSON MineField.json` dentro de uma pasta `Campo-Minado-Infinito`, essa pasta por sua vez fica em uma área especial do computador do jogador para poder ser acessada em toda sessão:

```

1  // Em src/MineFieldSerializer.cs:
2  namespace CampoMinado.Saves;
3
4  /// <summary>
5  /// Classe responsável pela serialização do mapa.
6  /// </summary>
7  public sealed class MineFieldSerializer
8  {
9      /// <summary>
10     /// Privado para não haver perigo de haver duas instâncias
11     /// </summary>
12     private MineFieldSerializer(MineField field)
13     {
14         Field = field;
15     }
16
17     /// <summary>
18     /// Única instância da classe
19     /// </summary>
20     private static MineFieldSerializer? s_instance;

```

```
21
22  /// <summary>
23  /// 0 campo que será serializado e deserializado
24  /// </summary>
25  public MineField Field { get; private set; }
26
27  private readonly JsonSerializerOptions Options = new() {
28      WriteIndented = false };
29
30  /// <summary>
31  /// Path do arquivo JSON que armazena o estado do campo minado.
32  /// </summary>
33  private readonly string Filename = Path.Combine(
34      Environment.GetFolderPath(Environment.SpecialFolder.
35          LocalApplicationData),
36      "Campo-Minado-Infinito",
37      "MineField.json"
38  );
39
40  /// <summary>
41  /// Retorna a instância da classe.
42  /// Caso não houver uma instância, cria-a, se tiver retorna a
43  /// instância criada na primeira vez
44  /// </summary>
45  public static MineFieldSerializer GetInstance(MineField field)
46  { ... }
47
48  /// <summary>
49  /// Serializa o mapa, adicionando ele no arquivo JSON MineField
50  /// .json
51  /// </summary>
52  /// <exception cref="InvalidOperationException">
53  /// Lançada caso o serializador não tenha sido inicializado.
54  /// </exception>
55  public static void Serialize() { ... }
56
57  /// <summary>
58  /// Desserializa o campo do mapa e exclui ele do arquivo JSON
59  /// </summary>
60  /// <returns>
61  /// Retorna o Campo do mapa que estava serializado.
62  /// </returns>
```

```

58  /// <remarks>
59  /// Retorna <see cref="null"> caso o arquivo não exista ou caso
    ele esteja vazio.
60  /// </remarks>
61  /// <exception cref="InvalidOperationException">
62  /// Lançada caso o serializador não tenha sido inicializado.
63  /// </exception>
64  public static MineField? Deserialize() { ... }
65 }

```

Os métodos `Serialize()` e `Deserialize()` servem, respectivamente, para salvar e carregar as informações do campo minado.

4 Exceções Tratadas

No nosso código, existiram 3 estados de erro notáveis que nos fizeram implementar *exceptions*.

4.1 InvalidBombDensityException

Essa exceção simboliza a inicialização do campo minado com uma densidade de bombas inválida, e é lançada quando o construtor de `MineField` recebe um valor fora do intervalo $[0, 1]$ para `bombsDensity`. A definição do construtor de `MineField` - e de algumas propriedades e campos da classe - é a que se segue:

```

1  // Em src/MineField.cs:
2  using CampoMinado.Core.Exceptions;
3
4  namespace CampoMinado.Core;
5
6  /// <summary>
7  /// Representa o mapa do campo minado. Armazena o estado de todas
    as células inicializadas
8  /// do mapa, encapsulando todos os detalhes de implementação.
9  /// </summary>
10 public class MineField
11 {
12     /// <summary>
13     /// Quantidade padrão de bombas por célula.
14     /// </summary>
15     public const double DEFAULT_BOMB_DENSITY = 0.3;
16

```

```

17  /// <summary>
18  /// Quantidade de bomba por célula definida para o campo minado
19  /// </summary>
20  public double BombDensity { get; }
21
22  /// <param name="bombDensity">
23  /// Densidade de bomba, deve estar no intervalo [0, 1].
24  /// </param>
25  /// <exception cref="InvalidBombDensityException">
26  /// Lançada se a densidade estiver fora de [0, 1].
27  /// </exception>
28  public MineField(double bombDensity = DEFAULT_BOMB_DENSITY)
29  {
30      if (bombDensity < 0.0 || bombDensity > 1.0)
31          throw new InvalidBombDensityException(bombDensity);
32
33      BombDensity = bombDensity;
34  }
35 }

```

Já a definição da exceção em si é a que se segue:

```

1  // Em src/Exceptions/InvalidBombDensityException.cs:
2  namespace CampoMinado.Core.Exceptions;
3
4  /// <summary>
5  /// Lançada quando a densidade de bombas especificada para o <see
6  ///   cref="MineField"/>
7  ///   está fora do intervalo [0, 1].
8  /// </summary>
9  public class InvalidBombDensityException(double attemptedValue)
10     : Exception(
11         $"Densidade de bombas inválida: {attemptedValue}. Esperado
12         valor entre 0 e 1."
13     )
14 { }

```

4.2 InvalidCellOperationException

Essa exceção simboliza a tentativa de uma operação que vai contra as regras do campo minado em uma célula. É lançada por `Cell` quando tenta-se revelar uma célula

com uma bandeira ou colocar uma bandeira em uma célula revelada:

```
1 // Em src/Cell.cs:
2 using CampoMinado.Core.Exceptions;
3
4 namespace CampoMinado.Core;
5
6 /// <summary>
7 /// Representa uma célula genérica ou não inicializada do campo
8   minado.
9 /// </summary>
10 public abstract record class Cell
11 {
12     /// <summary>
13     /// Retorna verdadeiro caso a célula tenha uma bandeira tenha
14     /// sido colocada pelo jogador
15     /// falso caso contrário.
16     /// </summary>
17     public bool HasFlag { get; private set; }
18
19     /// <summary>
20     /// Retorna verdadeiro caso a célula tenha sido revelada pelo
21     /// jogador, ou falso caso
22     /// contrário.
23     /// </summary>
24     public bool IsRevealed { get; private set; }
25
26     /// <summary>
27     /// Garante que uma bandeira estará nessa célula.
28     /// </summary>
29     /// <exception cref="InvalidCellOperationException">
30     /// Lançada se a célula já tiver sido revelada.
31     /// </exception>
32     public void EnsureFlag()
33     {
34         if (IsRevealed)
35             throw new InvalidCellOperationException(
36                 "Não é possível colocar bandeira em uma célula já
37                 revelada."
38             );
39
40         HasFlag = true;
41     }
42 }
```

```

38
39  /// <summary>
40  /// Garante que nenhuma bandeira estará nessa célula.
41  /// </summary>
42  public void EnsureNoFlag()
43  {
44      HasFlag = false;
45  }
46
47  /// <summary>
48  /// Marca a célula como revelada para o jogador.
49  /// </summary>
50  /// <exception cref="InvalidCellOperationException">
51  /// Lançada se a célula ainda tiver bandeira.
52  /// </exception>
53  public void Reveal()
54  {
55      if (HasFlag)
56          throw new InvalidCellOperationException(
57              "Não é possível revelar uma célula que ainda possui
58              bandeira."
59          );
60      IsRevealed = true;
61  }
62
63  public Cell() { }
64  }

```

Essa é a definição completa de `Cell`. A definição da exceção em si é a seguinte:

```

1  // Em src/Exceptions/InvalidCellOperationException.cs:
2  namespace CampoMinado.Core.Exceptions;
3
4  /// <summary>
5  /// Lançada quando uma operação sobre uma célula viola as regras
6  /// do jogo
7  /// (ex.: revelar célula com bandeira, ou colocar bandeira em cé
8  /// lula já revelada).
9  /// </summary>
10 /// <param name="message">
11 /// Mensagem que explica qual foi a operação inválida.
12 /// </param>

```

```

11 public class InvalidCellOperationException(string message) :
    Exception(message) { }

```

4.3 InvalidNearBombsException

Essa exceção é lançada toda vez que o setter de `SafeCell.NearBombs` recebe uma quantidade inválida de células adjacentes com bombas, isso é, menor que 0 ou maior que 8:

```

1  // Em src/SafeCell.cs:
2  using CampoMinado.Core.Exceptions;
3
4  namespace CampoMinado.Core;
5
6  /// <summary>
7  /// Representa uma célula sem bomba do campo minado. Armazena a
8  /// quantidade células com
9  /// bomba adjacentes.
10 /// </summary>
11 /// <param name="nearBombs">
12 /// Representa a quantidade de bombas adjacentes. É zero por
13 /// padrão.
14 /// </param>
15 public record class SafeCell(int nearBombs = 0) : Cell
16 {
17     /// <summary>
18     /// Representa a quantidade de células com bomba perto dessa cé-
19     /// lula.
20     /// </summary>
21     /// <remarks>
22     /// DEVE ser um número entre 0 e 8, afinal não tem como haver
23     /// uma quantidade negativa de
24     /// células adjacentes ou uma quantidade maior que 8.
25     /// </remarks>
26     /// <exception cref="InvalidNearBombsException">
27     /// Lançada ao atribuir um valor fora do intervalo [0, 8].
28     /// </exception>
29     public int NearBombs
30     {
31         get;
32         set
33         {
34             if (value < 0 || value > 8)

```

```

31         throw new InvalidNearBombsException(value);
32
33         field = value;
34     }
35     } = nearBombs;
36 }

```

A exceção `InvalidNearBombsException` é definida como:

```

1  // Em src/Exceptions/InvalidNearBombsException.cs:
2  namespace CampoMinado.Core.Exceptions;
3
4  /// <summary>
5  /// Lançada quando a quantidade de bombas adjacentes de uma <see
6  /// cref="SafeCell"/>
7  /// está fora do intervalo válido [0, 8].
8  /// </summary>
9  public class InvalidNearBombsException(int attemptedValue)
10     : Exception(
11         $"Quantidade de bombas adjacentes inválida: {attemptedValue}.
12         Esperado valor entre 0 e 8."
13     )
14 { }

```

5 Padrão de projeto escolhido

Nós usamos o padrão de Singleton na definição da classe `MineFieldSerializer`, visto que o salvamento e carregamento dos dados do jogador é única. Isso pode ser visto no snippet de código a seguir:

```

1  // Em src/MineFieldSerializer.cs:
2  namespace CampoMinado.Saves;
3
4  /// <summary>
5  /// Classe responsável pela serialização do mapa.
6  /// </summary>
7  public sealed class MineFieldSerializer
8  {
9      /// <summary>
10     /// Privado para não haver perigo de haver duas instâncias
11     /// </summary>
12     private MineFieldSerializer(MineField field)

```

```
13 {
14     Field = field;
15 }
16
17 /// <summary>
18 /// Única instância da classe
19 /// </summary>
20 private static MineFieldSerializer? s_instance;
21
22 /// <summary>
23 /// Retorna a instância da classe.
24 /// Caso não houver uma instância, cria-a, se tiver retorna a
25 /// instância criada na primeira vez
26 /// </summary>
27 public static MineFieldSerializer GetInstance(MineField field)
28 {
29     if (s_instance == null)
30     {
31         s_instance = new MineFieldSerializer(field);
32     }
33
34     return s_instance;
35 }
```

O campo `s_instance` armazena a instância atualmente definida, e ele é usado pelos métodos `Serialize()` e `Deserialize()`; `s_instance` pode ser acessado com `GetInstance(...)`.

6 Dificuldades e soluções

Uma das grandes dificuldades foi atender aos conceitos que eram obrigatórios de serem trabalhados, visto que isso deixava o nosso tema ainda mais complexo. Implementar especificamente o uso de exceções em um código que não tem muitos estados de erro, foi o mais difícil, mas nós conseguimos perceber estados inválidos que podiam ter exceções implementadas.

Outra coisa que foi difícil de fazer foi a serialização/deserialização em JSON, visto que muitas das coisas que existem no C# não são serializáveis: matrizes definidas com `[,]`, `ValueTuples` como chave de dicionários etc. Muitas coisas tiveram que ser adaptadas para possibilitar a serialização.

Além disso, antes de decidimos usar o Monogame que nos trouxe a relação de

herança e polimorfismo entre `MainGame` e `Game`, nós tivemos certa dificuldade em pensar em relações de herança até vir à mente a ideia de células "seguras" e "perigosas".

7 Conclusão e aprendizado individual

O jogo foi implementado com sucesso. Gostaríamos de ter colocado um sistema de pontuação e sistema de múltiplos saves, mas isso não foi possível. Outra coisa que pensamos em fazer era aumentar a qualidade do jogo em questão de interface, mas não implementamos nenhum tipo de menu, pause etc.

Aqui compilamos as observações individuais em relação ao trabalho.

7.1 Eduardo Lamounier

Eu achei muito bom realmente trabalhar com uma engine focada em desenvolvimento de jogos como o Monogame ao invés de Processing. Além disso, diferentemente de outras como Roblox Studio ou até mesmo Unity, Monogame mantém você com um pensamento de baixo nível enquanto programa e também não te obriga a usar outra IDE totalmente diferente.

Além disso, eu aprendi **muita** coisa de C# fazendo esse trabalho, mesmo que eu achasse que sabia bastante - porque eu havia tido certo contato com a linguagem antes.

7.2 Heitor Freitas

Ter a possibilidade de trabalhar no C#, que é uma linguagem nova para mim, é uma experiência única. Sendo uma linguagem com paradigma de orientação a objetos, com isso vários elementos abordados em sala de aula puderam ser vistos e implementados no algoritmo. Mesmo assim, ele se diferencia pela sua sintaxe mais "nova" em relação ao Java, isso implica em novas dificuldades que trabalhando junto aos meus colegas conseguimos resolver.

Enquanto ao MonoGame, é interessante o fato da Engine, não se distanciar muito do C#, permitindo uma maior facilidade e integração com o código.

No geral, esse trabalho me esclareceu muitas coisas sobre os padrões de construção, principalmente o *Singleton*, sobre alguns princípios de *POO* e para além do código, como trabalho em equipe, uso do git e github e o diagrama de classe (*UML*).

7.3 Adrian da Silva

Eu achei muito interessante trabalhar com o MonoGame e com o uso do C#. Eu aprendi uma linguagem nova e achei muito legal poder colocar em prática, o uso de *POO*

no trabalho. No começo eu tive um pouco de dificuldade para entender algumas partes do código, mas com o tempo fui conseguindo entender melhor como cada parte funcionava.

O Campo Minado Infinito foi um trabalho que eu achei bem diferente dos outros trabalhos que já fiz. Uma das partes mais difíceis foi fazer a parte das imagens e também a interação do mouse com o mapa. Foi necessário entender como fazer o mouse identificar as células e como fazer o jogo responder quando o jogador clicasse nelas.

8 Manual do usuário

Para jogar, você pode:

- Se movimentar pelo campo com as teclas WASD;
- Zoom-in pressionando a tecla I, zoom-out pressionando a tecla O;
- Revelar células com o botão esquerdo do mouse;
- Colocar/Retirar a bandeira de uma célula;
- Salvar o estado atual do campo minado com F4;
- Retomar o estado do campo minado anteriormente salvo com F5;
- Finalizar o programa pressionando ESC.